component inventory for 5-components

Below is the **component inventory for 5-components**, mapped to **one SCSS file per component block**, based on the HTML files you uploaded.

I am treating each **component root block** (`.c-*`) as the unit that should usually get **one SCSS file**. I am **not** including objects (`.o-*`), layouts (`.l-*`), or utilities (`.u-*`) in this inventory. The HTML files that drive this mapping are `header.html` , `navigation.html` , `mobile-menu.html` , `social-links.html` , `theme-toggle.html` , `hero.html` , `post-card.html` , `post-share.html` , `post-navigation.html` , `related-posts.html` , `footer.html` , `page.html` , `post.html` , `index.html` , and `projects.html` .

Already written

These are clearly present in the HTML and already correspond to real component blocks:

- `_button.scss` → `.c-button`
- `_mobile-menu.scss` → `.c-mobile-menu`
- `_navigation.scss` → `.c-nav` , `.c-nav-toggle`
- `_post-share.scss` → `.c-post-share`
- `_social-links.scss` → `.c-social-links` , plus shared link styling if you chose to include it there
- `_theme-toggle.scss` → `.c-theme-toggle`

Remaining component inventory: one SCSS file per block

Global/header/footer components

- `_brand.scss`
  Block: `.c-brand`
  Elements:
  - `.c-brand__logo`
  - `.c-brand__text`
  - `.c-brand__name`

    - `.c-brand__role`

      Used in `header.html`.
- `_site-header.scss`

  Block: `.c-site-header`

  Used in `header.html`. This file should style the **header shell only**, not navigation/theme/social internals.
- `_site-footer.scss`

  Block: `.c-site-footer`

  Elements:
    - `.c-site-footer__brand-link`
    - `.c-site-footer__tagline`
    - `.c-site-footer__copyright`

      Used in `footer.html`.
- `_icon-link.scss`

  Block: `.c-icon-link`

  Elements:
    - `.c-icon-link__icon`

      Used by `social-links.html` as a reusable leaf component. This is worth its own file because it is a standalone block reused across contexts.

Homepage / section / archive components

- `_hero.scss`

  Block: `.c-hero`

  Elements:
    - `.c-hero__eyebrow`
    - `.c-hero__title`
    - `.c-hero__description`
    - `.c-hero__actions`
    - `.c-hero__media`
    - `.c-hero__image`

      Used in `hero.html`.
- `_post-card.scss`

  Block: `.c-post-card`

  Elements:
    - `.c-post-card__inner`
    - `.c-post-card__media`
    - `.c-post-card__image-link`

- `.c-post-card__image`
- `.c-post-card__content`
- `.c-post-card__tags`
- `.c-post-card__tag`
- `.c-post-card__title`
- `.c-post-card__title-link`
- `.c-post-card__excerpt`
- `.c-post-card__meta`
- `.c-post-card__author`
- `.c-post-card__avatar`
- `.c-post-card__meta-text`
- `.c-post-card__author-name`
- `.c-post-card__meta-secondary`
- `.c-post-card__date`
- `.c-post-card__read-time`
- `.c-post-card__icon`

  Used in `post-card.html`, and reused by `index.html`, `projects.html`, `tag-page.html`, and `related-posts.html`.

- `_section-header.scss`

  Block: `.c-section-header`

  Elements:

  - `.c-section-header__title`

    Used in `index.html` and `related-posts.html`.

- `_section-cta.scss`

  Block: `.c-section-cta`

  Used in `index.html`. This can stay tiny, but it is still a distinct component wrapper in the markup.

- `_related-posts.scss`

  Block: `.c-related-posts`

  Used in `related-posts.html`. This should own the section shell spacing only, while `c-section-header` and `c-post-card` remain separate.

- `_page-header.scss`

  Block: `.c-page-header`

  Elements:

  - `.c-page-header__title`
  - `.c-page-header__subtitle`

    Used in `projects.html`.

- `_projects-filter.scss`
  Block: `.c-projects-filter`
  Elements:
  - `.c-projects-filter__inner`
  - `.c-projects-filter__chip`
  - `.is-active` as chip state

    Used in `projects.html`.
- `_projects-archive.scss`
  Block: `.c-projects-archive`
  Used in `projects.html`. This is mainly the archive section shell.

Page/post/article components

- `_article.scss`
  Block: `.c-article`
  Elements:
  - `.c-article__media`
  - `.c-article__image`
  - `.c-article__title`
  - `.c-article__subtitle`
  - `.c-article__meta`
  - `.c-article__author`
  - `.c-article__meta-sep`
  - `.c-article__date`
  - `.c-article__divider`

    Used in `page.html`.
- `_tag.scss`
  Block: `.c-tag`
  Used in `page.html`. Keep this separate if you want a reusable generic tag/pill component.
- `_post-cover.scss`
  Block: `.c-post-cover`
  Elements:
  - `.c-post-cover__media`
  - `.c-post-cover__image`

    Used in `post.html`.
- `_post.scss`
  Block: `.c-post`
  Elements:

- `.c-post__header`
- `.c-post__tags`
- `.c-post__tag`
- `.c-post__title`
- `.c-post__subtitle`
- `.c-post__meta`
- `.c-post__meta-main`
- `.c-post__author-image`
- `.c-post__author-avatar`
- `.c-post__meta-details`
- `.c-post__author`
- `.c-post__meta-inline`
- `.c-post__meta-date`
- `.c-post__date`
- `.c-post__meta-separator`
- `.c-post__reading-time`
- `.c-post__meta-icon`
- `.c-post__meta-actions`
- `.c-post__share`
- `.c-post__content`

  Used in `post.html`.

- `_post-navigation.scss`

  Block: `.c-post-nav`

  Elements:

  - `.c-post-nav__item`
  - `.c-post-nav__item--prev`
  - `.c-post-nav__item--next`
  - `.c-post-nav__media`
  - `.c-post-nav__content`
  - `.c-post-nav__label`
  - `.c-post-nav__label--prev`
  - `.c-post-nav__label--next`
  - `.c-post-nav__title`

  Used in `post-navigation.html`.

Final recommended 5-components inventory

Putting it all together, your 5-components layer should look like this:

Already written

- `_button.scss`
- `_mobile-menu.scss`
- `_navigation.scss`
- `_post-share.scss`
- `_social-links.scss`
- `_theme-toggle.scss`

Still to write

- `_brand.scss`
- `_site-header.scss`
- `_site-footer.scss`
- `_icon-link.scss`
- `_hero.scss`
- `_post-card.scss`
- `_section-header.scss`
- `_section-cta.scss`
- `_related-posts.scss`
- `_page-header.scss`
- `_projects-filter.scss`
- `_projects-archive.scss`
- `_article.scss`
- `_tag.scss`
- `_post-cover.scss`
- `_post.scss`
- `_post-navigation.scss`

Two small recommendations

First, I would keep `_icon-link.scss` separate from `_social-links.scss`. In your markup, `.c-icon-link` is a true reusable block and `.c-social-links__item` is contextual styling layered on top of it.

Second, I would keep `_section-header.scss` separate from `_page-header.scss`. They are different patterns in the HTML: one is a small reusable section heading, the other is a larger archive/page-intro block.

Recommended import order for `5-components.scss`

Use this order:

```scss
// ----------------------------------------------------------------------------
// 5-components manifest
// ITCSS layer: 5-components
//
// Rules:
// - Import low-dependency, reusable leaf components first
// - Import contextual/global shell components next
// - Import larger composite/content components later
// - Keep related components grouped together
// ----------------------------------------------------------------------------

@use "5-components/button";
@use "5-components/icon-link";
@use "5-components/social-links";
@use "5-components/theme-toggle";

@use "5-components/brand";
@use "5-components/navigation";
@use "5-components/mobile-menu";
@use "5-components/site-header";
@use "5-components/site-footer";

@use "5-components/tag";
@use "5-components/section-header";
@use "5-components/section-cta";
@use "5-components/page-header";
@use "5-components/projects-filter";
@use "5-components/projects-archive";

@use "5-components/hero";
@use "5-components/post-card";
@use "5-components/related-posts";

@use "5-components/article";
@use "5-components/post-cover";
@use "5-components/post-share";
@use "5-components/post-navigation";
@use "5-components/post";
```

Why this order works

It follows a practical dependency order:

1. **Leaf UI controls first**
   These are small reusable pieces:
   - `button`
   - `icon-link`
   - `social-links`
   - `theme-toggle`
2. **Global chrome next**
   Header/footer/navigation often compose those smaller controls:
   - `brand`
   - `navigation`
   - `mobile-menu`
   - `site-header`
   - `site-footer`
3. **Shared section/page patterns next**
   These are lightweight structural presentation components:
   - `tag`
   - `section-header`
   - `section-cta`
   - `page-header`
   - `projects-filter`
   - `projects-archive`
4. **Homepage/archive composites after that**
   These are bigger content cards/sections:
   - `hero`
   - `post-card`
   - `related-posts`
5. **Article/post-specific components last**
   These are the most context-heavy:
   - `article`
   - `post-cover`
   - `post-share`
   - `post-navigation`
   - `post`

That keeps the manifest readable and reduces the chance that a big component appears before the smaller components it visually depends on.

Priority order for writing the remaining files

I would write them in this order:

Tier 1: write first

These affect the biggest visible parts of the site and are reused most.

1. `_icon-link.scss`
2. `_brand.scss`
3. `_site-header.scss`
4. `_site-footer.scss`
5. `_hero.scss`
6. `_post-card.scss`

Why Tier 1 first

- `icon-link` supports your social link system cleanly.
- `brand`, `site-header`, and `site-footer` define the global frame of the site.
- `hero` and `post-card` drive the homepage and archive presentation.

Tier 2: write next

These complete shared page/archive patterns.

7. `_section-header.scss`
8. `_section-cta.scss`
9. `_page-header.scss`
10. `_projects-filter.scss`
11. `_projects-archive.scss`
12. `_related-posts.scss`

Why Tier 2 next

These files are important, but usually smaller and easier once your global visual language is already set by Tier 1.

Tier 3: write last

These are more page-specific and usually benefit from the earlier styles already being established.

13. `_tag.scss`
14. `_article.scss`
15. `_post-cover.scss`
16. `_post-navigation.scss`
17. `_post.scss`

Why Tier 3 last

These are tightly tied to content templates like `page.html` and `post.html`, so they are easier to design after the reusable global and archive components are stable.

My recommended actual build sequence

If you want the most efficient path, do it in this sequence:

1. `_icon-link.scss`
2. `_brand.scss`
3. `_site-header.scss`
4. `_site-footer.scss`
5. `_hero.scss`
6. `_post-card.scss`
7. `_section-header.scss`
8. `_page-header.scss`
9. `_projects-filter.scss`
10. `_related-posts.scss`
11. `_tag.scss`
12. `_article.scss`
13. `_post-cover.scss`
14. `_post-navigation.scss`

15. `_post.scss`
16. `_section-cta.scss`
17. `_projects-archive.scss`

I moved `section-cta` and `projects-archive` toward the end because they are often very small wrapper components.

One small adjustment to keep in mind

If your `.c-site-header` file ends up containing only spacing/layout around object-level children, you may discover that some of that belongs in `4-objects` instead. But starting with `_site-header.scss` in components is still sensible because your HTML clearly uses it as a component shell.

Here is the final condensed version:

```
Import order:
button
icon-link
social-links
theme-toggle
brand
navigation
mobile-menu
site-header
site-footer
tag
section-header
section-cta
page-header
projects-filter
projects-archive
hero
post-card
related-posts
article
post-cover
post-share
post-navigation
post
```

```
Write first:
icon-link
brand
site-header
site-footer
hero
post-card
section-header
page-header
projects-filter
related-posts
tag
article
post-cover
post-navigation
post
section-cta
projects-archive
```

Next, I can turn this into a **starter scaffold for each missing SCSS file**, with the standard header comments and empty block/element structure.